Api Reference

A CEREBRUM Research Publication (v1.2.4)

Bryan Alexander Buchorn
Independent Researcher

Abstract

Overview: This guide provides a conceptual entry point into the CEREBRUM framework. It is designed for practitioners and researchers seeking to understand the core reasoning signals of Framework Implementation Guide.

0.1 CEREBRUM REST API Reference

Base URL: `http://localhost:8200` **API Version**: v1.1.0 **Authentication**: JWT Bearer token (all endpoints except `/health`)

—

Authentication

All endpoints (except GET `/health`) require a JWT Bearer token:

```
Authorization: Bearer <jwt_token>
```

Tokens are generated using HMAC-SHA256 with the CEREBRUM_JWT_SECRET environment variable. Token lifetime defaults to 3600 seconds; configure via CEREBRUM_JWT_TTL.

Error response (401 Unauthorized):

```
{"detail": "Missing or invalid authorization token"}
```

—

Endpoints

Core Reasoning

`POST /query` Execute a multi-hop reasoning query against the loaded graph.

Request body:

```
{
    "entity": "Marie Curie",
    "max_hops": 3,
    "beam_width": 5,
    "top_k": 10,
    "probabilistic": false,
    "warm_start_strength": 0.0,
    "community_snapshot": true
}
```

Field	Type	Default	Description
`entity`	string	required	Starting entity for traversal
`max_hops`	int	3	Maximum reasoning depth
`beam_width`	int	5	Number of candidates per hop
`top_k`	int	10	Number of answer paths to return
`probabilistic`	bool	false	Enable Bayesian beam search
`warm_start_strength`	float	0.0	First-hop Beta prior seeding (Phase 19)
`community_snapshot`	bool	true	Enable query snapshot isolation (Phase 20)

Response (200 OK):

```
{
    "query_entity": "Marie Curie",
    "answers": [
        {
            "entity": "Radioactivity",
            "score": 0.847,
            "path": ["Marie Curie", "discovered", "Polonium", "property_of", "
               Radioactivity"],
            "path_confidence": 0.823,
            "confidence_interval": [0.78, 0.87],
            "communities": [2, 2, 5],
            "hmac": "sha256:a3f4b2..."
        }
    ],
    "traversal_ms": 6.3,
    "hops_explored": 847,
    "snapshot_id": "snap_1743000000"
}
```

curl example:

```
curl -X POST http://localhost:8200/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"entity": "Marie Curie", "max_hops": 3}'
```

—

`GET /query` Simple GET-style query for integrations that cannot send POST bodies.
Query parameters: `entity` (required), `max_hops` (default: 3), `top_k` (default: 10)
Response: Same schema as `POST /query`.
—

Graph Information

`GET /communities` Return the current community partition of the loaded graph.
Response (200 OK):

```
{
    "algorithm": "DSCF",
    "modularity_q": 0.432,
    "num_communities": 7,
    "communities": {
        "0": ["Marie Curie", "Pierre Curie", "Polonium", "Radium"],
        "1": ["Einstein", "Special Relativity", "General Relativity"],
        "2": ["Newton", "Gravity", "Calculus"]
    },
    "computed_at": 1743000000
}
```


—

`GET /graph/stats` Return structural statistics for the loaded graph.

Response (200 OK):


```
{
    "num_nodes": 21,
    "num_edges": 30,
    "avg_degree": 2.86,
    "density": 0.143,
    "num_communities": 7,
    "modularity_q": 0.432,
    "avg_clustering_coefficient": 0.41
}
```


—

`GET /graph/entity/entity_id` Return all edges and community membership for a specific entity.

Path parameter: `entity_id` — URL-encoded entity string

Response (200 OK):


```
{
    "entity": "Marie Curie",
    "community": 0,
    "soft_memberships": {"0": 0.72, "2": 0.18, "5": 0.10},
    "edges_out": [
        {"target": "Polonium", "relation": "discovered", "weight": 0.95},
        {"target": "Radium", "relation": "discovered", "weight": 0.93}
    ],
    "edges_in": [
        {"source": "Nobel_Prize_1903", "relation": "awarded_to", "weight":
           0.99}
    ],
    "pagerank": 0.043,
    "betweenness": 0.127,
    "degree": 5
}
```


—

Bridge Twins

`GET /bridges` Return all active bridge twin nodes.

Response (200 OK):

```
{
    "bridge_count": 3,
    "bridges": [
        {
            "twin_id": "bridge_7",
            "original_id": "Curie_Institute",
            "source_community": 0,
            "destination_community": 3,
            "crossing_count": 47,
            "ltp_weight": 0.84,
            "created_at": 1743000000
        }
    ]
}
```

—

`GET /bridges/twin_id` Return details for a specific bridge twin node.

Path parameter: `twin_id` — bridge twin identifier

Response: Single bridge object (same schema as items in `/bridges`).

—

Streaming

`GET /stream/events` Server-Sent Event stream of raw graph update events.

Headers:

```
Accept: text/event-stream
Authorization: Bearer <token>
```

Event format:

```
event: edge_added
data: {"u": "A", "v": "B", "relation": "CAUSES", "weight": 0.72, "timestamp":
    1743000000.0}

event: community_rebalanced
data: {"communities": 7, "q": 0.43, "pruned_bridges": 2}

event: bridge_formed
data: {"twin_id": "bridge_8", "src_community": 0, "dst_community": 3}
```

—

`GET /stream/insights` Server-Sent Event stream of materialized insight events.

Headers: Same as `/stream/events`.

Event format:

```
event: insight_link
data: {"source": "A", "target": "B", "score": 0.81, "state": "SPECULATIVE"}

event: insight_verified
data: {"source": "A", "target": "B", "fwd_conf": 0.78, "rev_conf": 0.71}

event: insight_refuted
data: {"source": "A", "target": "B", "reason": "bilateral_probe_failed"}
```

—

`POST /stream/push` Push a single event into the streaming ingest pipeline.

Request body:

```
{
    "subject": "sensor_42",
    "predicate": "READS",
    "object": "temperature_chamber_1",
    "timestamp": 1743000000.0,
    "weight": 0.85
}
```

Response (202 Accepted):

```
{"queued": true, "queue_depth": 14}
```

—

Administration

`GET /health` Health check endpoint. No authentication required.

Response (200 OK):

```
{
    "status": "healthy",
    "version": "1.1.0",
    "graph_loaded": true,
    "num_nodes": 21,
    "num_edges": 30,
    "uptime_s": 3600
}
```

—

`POST /admin/rebalance` Trigger an immediate synchronous DSCF community rebalance. Requires `scope: admin` in JWT.

Response (200 OK):

```
{
    "rebalanced": true,
    "new_q": 0.448,
    "communities": 7,
    "pruned_bridges": 1,
    "duration_ms": 480
}
```

—

`GET /admin/resource-stats` Return current ResourceGovernor metrics.

Response (200 OK):

```
{
    "cpu_percent": 12.4,
    "memory_mb": 384,
    "active_queries": 3,
    "queued_events": 47,
    "rebalance_count": 12,
    "last_rebalance_at": 1743000000
}
```

—

`DELETE /admin/cache` Clear the materialized path cache.

Response (200 OK):

```
{"cleared": true, "entries_removed": 1024}
```

—

Error Codes

HTTP Status	Code	Description
400	`INVALID_ENTITY`	Entity not found in graph
400	`INVALID_PARAMS`	Request parameter validation failed
401	`AUTH_MISSING`	Authorization header absent
401	`AUTH_EXPIRED`	JWT token has expired
401	`AUTH_INVALID`	JWT signature invalid
403	`SCOPE_INSUFFICIENT`	Token scope does not permit operation
404	`NOT_FOUND`	Requested resource not found
429	`RATE_LIMITED`	Request rate exceeded; retry after N seconds
500	`TRAVERSAL_FAILED`	Internal traversal error (check logs)
503	`GRAPH_NOT_LOADED`	No graph loaded; call /admin/load first

Error response format:

```
{
    "error": "INVALID_ENTITY",
    "detail": "Entity 'xyz' not found in graph",
    "status": 400
}
```

—

Rate Limiting

Default rate limits (configurable via environment variables):

Endpoint	Default Limit
`POST /query`	100 requests/minute
`GET /query`	100 requests/minute
`GET /communities`	30 requests/minute
`POST /stream/push`	1000 requests/minute
`GET /stream/events`	10 concurrent connections
`GET /stream/insights`	10 concurrent connections
`POST /admin/*`	10 requests/minute

Rate limit responses include:

```
Retry-After: 15
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1743000060
```

—

SDK Usage

```
# Python client (using httpx)
import httpx, os

client = httpx.Client(
    base_url="http://localhost:8200",
    headers={"Authorization": f"Bearer {os.environ['CEREBRUM_TOKEN']}"}
)

response = client.post("/query", json={"entity": "Marie Curie", "max_hops":
   3})
result = response.json()
print(result["answers"][0]["path"])
```